

Autonomous Space Debris Detection & Low-Delta V Collision Avoidance

1. Summary & Core Concept

Problem: Existing ground-based collision warnings rely on imprecise radar data with massive uncertainty bounds ("error ellipsoids"). As a result, satellites execute premature, high- Δv avoidance maneuvers for false positives, burning critical fuel and shortening their operational lifespan.

Solution: An autonomous, zero-added-sensor-mass edge computing pipeline. This architecture repurposes existing onboard Star Trackers and offloads image processing to a dedicated FPGA accelerator.

Operational Workflow

1. **Coarse Warning Ingestion:** The satellite receives ground-station Conjunction Data Messages (CDMs) flagging potential high-risk conjunctions weeks in advance.
2. **Predictive Alignment:** The spacecraft uses reaction wheels to slew its orientation, pointing an existing Star Tracker along the predicted threat approach vector during non-mission orbits.
3. **Parallel Edge Processing:** The raw Star Tracker video stream is tapped into an independent FPGA module to prevent hijacking the primary Attitude Determination and Control System (ADCS).
4. **Transient Streak Extraction:** The FPGA runs real-time frame differencing to strip static background stars and isolate high-velocity debris streaks.
5. **3D State Estimation:** An onboard Extended Kalman Filter (EKF) converts 2D angular streak data across multiple frames into a precise 3D trajectory.

6. **Intersection Verification:** The system checks if the computed 3D trajectory intersects the satellite's defined safety volume.
7. **Proactive Burn Execution:** If the collision probability exceeds the threshold, the satellite calculates and executes a minimal Δv maneuver at the optimal orbital node long before closest approach.

2. Active-Active Avoidance Protocol (Spacecraft vs. Spacecraft)

When a predicted conjunction involves two fully functional, maneuverable satellites, executing double maneuvers escalates the collision risk. The onboard system runs a multi-variable decision engine to determine which spacecraft executes the maneuver.

Decision Matrix & Scoring Function

Both spacecraft exchange minimal state packets via ground uplink or inter-satellite links. The system calculates a Maneuver Responsibility Score (SSS) based on four weighted metrics:

$$SSS = w_1 \cdot \text{Cost}_{\text{lifetime}} + w_2 \cdot \text{Cost}_{\Delta v} + w_3 \cdot \text{Cost}_{\text{priority}} + w_4 \cdot \text{Cost}_{\text{downtime}}$$

The satellite with the lower score performs the maneuver:

1. **Remaining Delta-V Reserves (Δv Margin):** The satellite with greater remaining fuel reserves assumes priority to execute the burn.
2. **Remaining Operational Lifetime:** A satellite near End-of-Life (EOL) preserves its remaining fuel; the younger asset with a longer operational margin takes the burn.
3. **Mission Priority / Payload Criticality:** High-value national defense or primary scientific payloads are prioritized to remain on track over secondary or commercial payloads.
4. **Operational Downtime / Recovery Latency:** Measures the time required for a satellite to re-orient, re-stabilize, and resume revenue/mission operations post-burn. The spacecraft with the shorter operational recovery time executes the burn.

3. Ground-Station AI Integration (Drag Prediction)

The Objective: Predict atmospheric drag fluctuations to tighten the error ellipsoid before uplinking the data to the satellite.

The Stack: The model is extremely lightweight. A massive Deep Neural Network (DNN) is not needed, as it takes too long to train and is prone to overfitting. The architecture utilizes a standard XGBoost regressor or a simple LSTM (Long Short-Term Memory) time-series model in Python.

The Inputs (The Features):

1. **Space Weather Data:** Pulls the F10.7 solar flux and Kp index (geomagnetic activity) APIs from NOAA. Solar flares expand the atmosphere and cause sudden drag.
2. **Historical SSA Data:** Pulls historical Two-Line Elements (TLEs) from Space-Track.org to track how specific debris orbits have degraded in the past.

The Output: The model generates a predictive drag coefficient. The Python dashboard uses this to calculate a refined Conjunction Data Message (CDM) with a significantly smaller error margin.

The Integration: The dashboard displays the 3D orbit and the "AI Risk Score". It then uplinks the refined CDM to the satellite. The Star Tracker and FPGA take that refined data, observe the actual physical space, and make the final deterministic burn decision.

4. Critical Edge Cases & Technical Mitigations

- **Eclipse / Solar Shadow (Optical Blindness):** Debris reflects sunlight; in Earth's shadow, optical detection fails.
 - *Mitigation:* The orbit propagator evaluates solar illumination vectors. If the conjunction occurs in darkness, the system automatically falls back to ground-based CDM data to execute a conservative burn.
- **High Relative Velocity & Streak Smearing:** Head-on approaches ($>15 \text{ km/s}$) turn point sources into faint, spread-out streaks that standard star algorithms discard as noise.

- *Mitigation:* The FPGA runs a line-detection pipeline (e.g., parallelized Hough transform) tuned specifically for multi-frame transient line detection.
- **Sensor Saturation (Sun/Earth Limb Blinding):** Pointing star trackers near the Sun or Earth's atmospheric limb washes out the sensor.
 - *Mitigation:* The slew planner checks solar/nadir exclusion zones before reorienting the spacecraft. If unviable, the maneuver defaults to ground estimates.
- **ADCS Computation Hijack:** Diverting primary MCU cycles to run heavy vision algorithms causes attitude lock drop.
 - *Mitigation:* Hardware isolation. Raw pixel feeds stream directly to an FPGA co-processor via dual-port RAM/DMA, leaving the ADCS flight MCU dedicated to attitude control.
- **2D-to-3D Depth Ambiguity:** A single optical sensor provides angular position, not direct range.
 - *Mitigation:* Implements Angles-Only Orbit Determination (AOOD) fused with a multi-frame EKF to resolve orbital elements and range over consecutive orbit passages.
- **Mission Slew Trade-off Cost:** Rotating the satellite to look for debris disrupts Earth-pointing primary payloads.
 - *Mitigation:* An algorithmic trade-off engine calculates if the revenue/data loss of a slew check is lower than the expected fuel cost of an unconfirmed blind burn.

5. Project Development Roadmap & Task Breakdown

Phase 1: Physics, Environment & Orbital Simulation Engine

- **Orbital Mechanics Core:** Implement a 2-body orbital propagator with J_2 Earth oblateness perturbations in Python.
- **Celestial Background Generator:** Build a static celestial sphere rendering cataloged star maps, constellations, Sun, and Moon vectors.

- **Lighting & Shadow Pipeline:** Calculate solar illumination vectors, Earth shadow (umbra/penumbra) boundaries, and glare limits.
- **Spacecraft & Sensor Kinematics:** Model reaction wheel torque, spacecraft slew rates, and Star Tracker Field of View (FOV) geometry ($15^\circ \times 15^\circ$ cone).

Phase 2: Star Tracker Optical Simulation & Sensor Modeling

- **Frame Generator:** Synthesize optical frame outputs replicating sensor resolution, dark current noise, exposure times, and integration blur.
- **Streak Dynamics Engine:** Render moving debris streaks based on relative velocity vectors, apparent magnitude, and integration time.

Phase 3: FPGA Hardware Co-Processor (Verilog/VHDL)

- **I/O Pipeline:** Design UART/SPI/Ethernet wrappers to stream simulated video frames into the physical FPGA dev board (Artix-7).
- **Frame Differencing Core:** Implement parallel line-buffer memory structures for high-speed background subtraction.
- **Feature Extraction Unit:** Build Verilog logic to extract centroid coordinates and streak vectors from raw pixel streams in real-time.
- **Control Bus Interface:** Output filtered streak data back to the primary system via AXI/SPI bus.

Phase 4: Core Algorithmic Suite

- **Angles-Only Orbit Determination (AOOD):** Implement Gauss/Laplace algorithms to convert line-of-sight angular vectors into preliminary orbit state vectors.
- **Extended Kalman Filter (EKF):** Write recursive state-estimation routines to refine debris range, velocity, and orbital parameters over time.
- **Collision Geometry Engine:** Compute distance of closest approach (DCA) and collision probability against the spacecraft's safety bounding box.
- **Active-Active Decision Engine:** Code the multi-variable scoring model for operational satellite-to-satellite burn negotiation.

- **Trajectory Optimization Routine:** Calculate minimum Δv impulsive burn vectors at optimal true anomaly locations.

Phase 5: Dataset & Test Automation

- **Synthetic Data Pipeline:** Generate structured testing datasets containing varying streak velocities, lighting conditions, background noise levels, and mock CDM inputs.
- **Hardware-in-the-Loop (HITL) Bench:** Link the Python simulation environment directly to the physical FPGA development board to run real-time hardware validation demos.

Here is the updated text for **Slide 2**.

I have added the live ground-based avoidance algorithm to the explanation and completely rewritten the "How it addresses the problem" section to relentlessly focus on the most critical aerospace metric: **Propellant (Δv) preservation**.

Detailed explanation of the proposed solution:

- **Ground-Segment AI (Drag Prediction):** An XGBoost/LSTM model ingests live space weather data (solar flux) to predict atmospheric drag fluctuations.
- **Error Ellipsoid Reduction:** The AI calculates a predictive drag coefficient, tightening the Conjunction Data Message (CDM) uncertainty bounds before uplink.
- **Onboard Edge-Vision:** An isolated FPGA processes raw Star Tracker video to visually confirm the debris trajectory in real-time without interrupting the main flight computer.
- **Live Ground-Based Avoidance Algorithm:** Confirmed 3D trajectory data is downlinked to the ground station, where a multi-variable decision engine calculates the absolute minimum Δv maneuver and resolves satellite-to-satellite right-of-way.

How it addresses the problem:

- **Protects the Most Critical Metric (Propellant / Δv):** Existing ground radar produces massive error ellipsoids, forcing operators to execute blind, premature maneuvers for false positives. Our edge-vision confirms actual threats, eliminating wasted fuel burns and directly extending the satellite's revenue-generating operational lifetime.
- **Resolves the "Double Burn" Hazard:** For operational satellite-to-satellite conjunctions, the live ground algorithm computes maneuver right-of-way based on mission priority, downtime cost, and remaining fuel reserves, preventing escalating collision risks.
- **Zero Hardware Penalty:** Achieves high-precision local tracking without adding new, heavy sensors to the payload mass budget.

Here is the exact division of labor for your team. This architecture is designed so that no one is blocked by anyone else. Each person builds their component as an isolated "black box" with agreed-upon inputs and outputs.

1. The Systems Architect & Physics Engine (You)

You own the "Ground Truth." If your numbers are wrong, the whole system fails.

- **Task 1 (Math):** Write the Python script using `skyfield` to propagate the orbits of a satellite and a piece of debris. Calculate their relative (X, Y, Z) vectors.
- **Task 2 (Synthetic Camera):** Use OpenCV to project those vectors onto a black 2D canvas. Draw the static stars and the moving debris streak.
- **Task 3 (Data Routing):** Stream the raw OpenCV pixels out of your USB port (PySerial) for the FPGA, and simultaneously stream the (X, Y, Z) JSON data to a local WebSocket for the frontend.

2. The Hardware Engineer (FPGA)

They treat your Python serial stream as if it were a physical camera sensor.

- **Task 1 (Ingestion):** Write the Verilog UART/SPI receiver to capture the incoming byte stream from your laptop.
- **Task 2 (Memory Routing):** Route the incoming frame into the FPGA's onboard PSRAM (line-buffering).
- **Task 3 (Frame Differencing):** Write the logic to subtract the previous frame from the current frame to erase the static stars.
- **Task 4 (Extraction):** Output a simple text or byte array containing the (X, Y) pixel coordinates of the detected debris streak.

3. The Machine Learning Engineer (Ground AI)

They do not need to know anything about the hardware or the 3D physics. They just process data.

- **Task 1 (Data Fetching):** Write a script to continuously pull the NOAA F10.7 solar flux API, Kp index, and Space-Track.org TLEs.
- **Task 2 (The Model):** Train an XGBoost or basic LSTM time-series model on historical data to correlate solar weather with atmospheric drag.
- **Task 3 (The Output):** Create a function that outputs a "Drag Multiplier" or a refined Error Ellipsoid radius based on the current space weather.

4. The Frontend Developer (Dashboard UI)

They do not touch physics or hardware. They only build the UI to impress the judges.

- **Task 1 (The Globe):** Set up a local web server running CesiumJS to render the 3D Earth.
- **Task 2 (Data Ingestion):** Connect to the WebSocket you built in Step 1 and parse your JSON payload.
- **Task 3 (Visualization):** Plot the satellite and debris on the globe. Draw a translucent 3D cone projecting from the satellite to represent the Star

Tracker's Field of View. Plot the AI's "Error Ellipsoid" as a glowing sphere around the debris.

5. The Project Manager (Your Friend)

While the four of you are coding, your PM protects the team from administrative friction.

- **Task 1:** Finalize the PPT, fill out all VNEST forms, and ensure the 6-member limit and female-member rules are strictly followed.
- **Task 2:** Write the pitch script and memorize the non-technical justifications (cost savings, fuel preservation, Kessler syndrome).
- **Task 3:** Manage the hackathon timeline. Force the team to integrate their parts 12 hours before the deadline.